



MediaEdge

Architecture Design Document (ADD)

Version 1.0

Author : Kartik Sharma

1. Project Overview

Project Name

MediaEdge

Tagline

A Spring Boot-based Hybrid Media Delivery Platform demonstrating modern video streaming using HLS, chunk-level caching, and a custom CDN Engine.

Introduction

MediaEdge is a backend-focused streaming platform designed to demonstrate how modern streaming services efficiently deliver media content.

Unlike traditional CRUD applications, MediaEdge focuses on solving backend engineering problems including:

- Efficient media streaming
- Chunk-based content delivery
- Hybrid CDN architecture
- Local edge caching
- Cache management
- Performance optimization

- Secure authentication
- Scalable software architecture

The application uses an anime-themed interface as a demonstration, while the primary goal of the project is to build a reusable **Media Delivery Engine**.

2. Project Objectives

The objectives of MediaEdge are:

- Learn how video streaming platforms work.
- Build a Hybrid CDN from scratch.
- Understand HLS streaming.
- Reduce origin requests through intelligent caching.
- Improve streaming performance using local edge cache.
- Learn Spring Boot architecture for large backend systems.
- Build a portfolio-quality backend project.

3. Functional Requirements

Guest Users

Guests can:

- Browse anime
- Search anime
- View anime details
- Stream episodes
- Change video quality

Guests cannot:

- Download episodes
- Save watch history
- Synchronize progress

Registered Users

Registered users can:

- Login/Register
- Continue Watching
- Watch History
- Download Episodes
- Manage Profile

4. Non-Functional Requirements

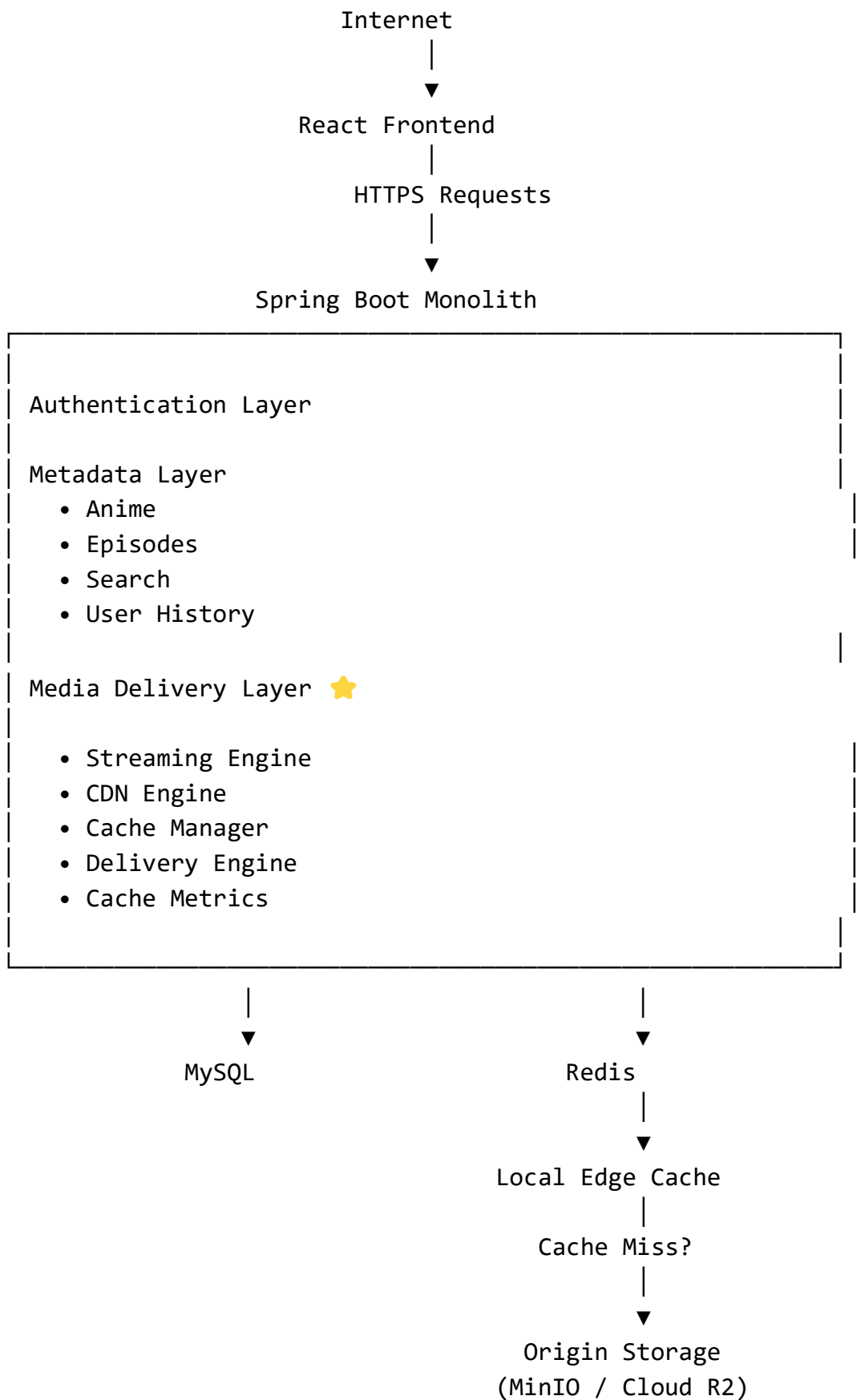
The system should provide:

- Low latency streaming
- High cache hit ratio
- Efficient storage usage
- Secure authentication
- Modular code structure
- Easy deployment using Docker
- Simple maintainability

5. Technology Stack

Layer	Technology
Language	Java 21
Framework	Spring Boot
Security	Spring Security + JWT
ORM	Spring Data JPA
Database	MySQL
Cache	Redis
Storage	MinIO
VideoProcessing	FFmpeg
Streaming	HLS
Frontend	React
Deployment	Docker Compose

6. High-Level Architecture



7. System Layers

Authentication Layer

Responsible for:

- User registration
- Login
- JWT generation
- Password hashing
- Download authorization

Guests may stream videos without authentication.

Metadata Layer

This layer manages all application data except video files.

Responsibilities:

- Anime
- Episodes
- Search
- Genres
- Continue Watching
- Watch History

Uses:

- MySQL
- Redis

Media Delivery Layer

This is the core of MediaEdge.

It contains all media-related logic.

Modules include:

Streaming Engine

Responsible for:

- Serving HLS playlists
- Adaptive streaming
- Video quality selection

CDN Engine

Responsible for:

- Chunk lookup
- Cache hit detection
- Cache miss handling
- Origin communication

Cache Manager

Responsible for:

- Local storage
- LRU eviction
- TTL expiration
- Disk usage

Delivery Engine

Responsible for:

- Delivering video chunks
- Downloading missing chunks
- Serving cached media

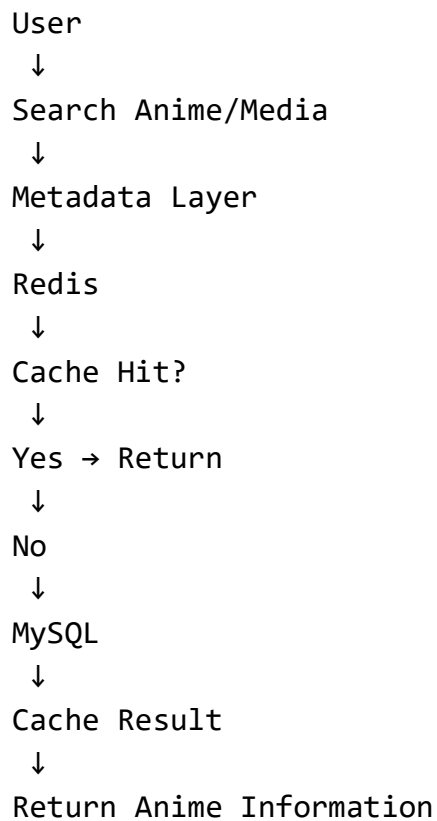
Metrics Collector

Responsible for:

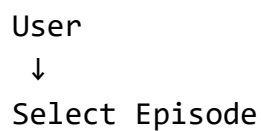
- Cache hit ratio
- Cache misses
- Origin requests
- Response time

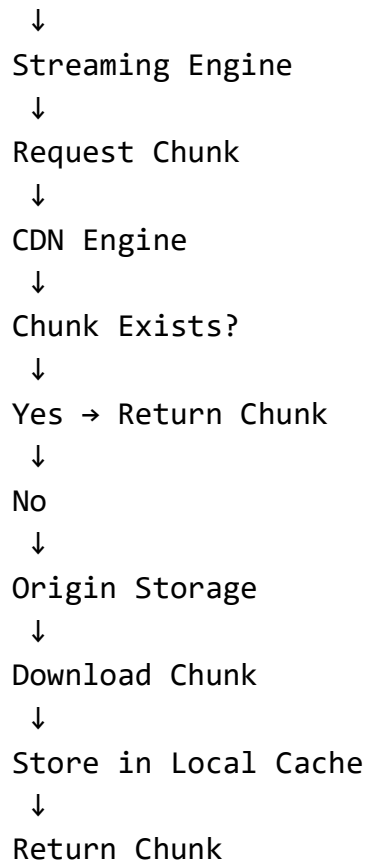
8. Request Flow

Search Flow

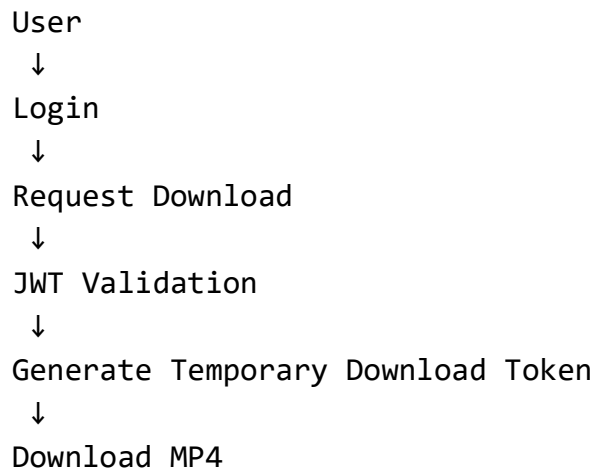


Streaming Flow





Download Flow



9. CDN Design

MediaEdge implements a **Pull-Based Hybrid CDN**.

Cache Hit

If the requested chunk is already cached:

- Return immediately.

Cache Miss

If the requested chunk is unavailable:

- Retrieve it from the origin.
- Store it in the local cache.
- Return it to the user.

Cache Eviction

Version 1 uses:

- LRU (Least Recently Used)

Future versions may support:

- LFU
- TTL-based eviction
- Hybrid policies

10. Data Storage

MySQL

Stores:

- Users
- Anime
- Episodes
- Watch History

Redis

Stores:

- Search cache
- Metadata cache
- Continue Watching cache
- CDN metadata
- Temporary download tokens

Local Edge Cache

Stores:

- .m3u8
- .ts
- Cache metadata

Origin Storage

Stores:

- Original MP4
- Generated HLS playlists
- HLS chunks

11. Security

MediaEdge implements:

- BCrypt password hashing
- JWT authentication
- HTTPS (production)

Guests can stream media.

Only authenticated users may download episodes or synchronize history.

12. Future Enhancements

Version 2 may include:

- Multiple edge nodes
- Redis Pub/Sub cache invalidation
- Regional edge routing
- Prometheus & Grafana
- AI recommendations
- Cloud deployment
- Kubernetes

13. Conclusion

MediaEdge is designed as a backend engineering project that demonstrates real-world media delivery concepts rather than focusing on UI complexity.

The project emphasizes:

- Clean software architecture
- Efficient content delivery
- Hybrid CDN design
- Chunk-level caching
- HLS streaming
- Performance optimization
- Secure authentication
- Modern Java backend development

